

Contexto Global del Proyecto: Cretania

Documento de inicialización para asistentes de código AI (Cursor, Windsurf) |
Entorno MySQL

1. Visión General del Proyecto

Cretania es una red de servidores de Minecraft modificado. El objetivo actual es desarrollar un sistema de sincronización de datos de jugador (inventario, vida, experiencia y datos de mods) transparente y sin interrupciones entre múltiples servidores backend.

Directriz Principal: La solución debe ser 100% "Server-Side". Los jugadores no deben requerir instalar ningún mod en sus clientes para que la sincronización funcione.

2. Entorno Tecnológico

- **Backend:** Servidores NeoForge.
- **Proxy:** Velocity.
- **Almacenamiento:** Base de Datos **MySQL (o MariaDB)**. Es obligatorio utilizar un pool de conexiones asíncrono como **HikariCP** para evitar colapsar el hilo principal (Main Thread) del servidor durante las consultas.
- **Mods a tener en cuenta:** El servidor cuenta con mods de alta complejidad técnica como *Create* y *TerraFirmaCraft*. El guardado no puede limitarse al inventario estándar; debe abarcar el NBT completo (**CompoundTag**) para conservar *Capabilities*.

3. Estructura del Espacio de Trabajo Recomendada

Para mantener el código organizado, se requiere construir esto como un proyecto multi-módulo (o dos proyectos separados):

- **cretania-neoforge/** : Contiene el mod backend. Se encarga de serializar el NBT a Base64, guardarlo en MySQL de forma asíncrona, bloquear al jugador y enviar el feedback visual.
- **cretania-velocity/** : Contiene el plugin del proxy. Actúa como un semáforo de estados escuchando el *Plugin Messaging Channel*.

4. Instrucciones para la IA (Prompt de Inicialización)

INSTRUCCIONES DE SISTEMA PARA EL ASISTENTE DE CÓDIGO:

1. Actúa como un desarrollador experto en Java, NeoForge API y Velocity API.
2. Este proyecto se llama "Cretania". Estás desarrollando un sistema de sincronización "Server-Side" usando MySQL.
3. Tienes a tu disposición un documento técnico llamado "Especificacion_Maestra_Cretania_MySQL.pdf". DEBES basar la arquitectura, nombres de canales de red (cretania:sync) y eventos de bloqueo estrictamente en ese documento.
4. Tu primera tarea es ayudar al usuario a configurar los archivos `build.gradle` para ambos módulos, incluyendo la dependencia del conector de MySQL, HikariCP (para el mod) y la API de Velocity.
5. Evita código "cliente-side". No uses eventos que solo existan en `net.neoforged.api.distmarker.Dist.CLIENT`.
6. El código debe ser limpio, modular y estar bien comentado. TODAS las consultas a MySQL deben manejarse en hilos separados (ej. `CompletableFuture.runAsync()`) para no afectar los TPS del servidor.

5. Manejo de Errores y Casos Límite

- Si el servidor de destino no tiene conexión a MySQL, el jugador debe ser desconectado (Kick) de forma segura en lugar de sobrescribir su inventario con datos vacíos.
- Asegurar que el campo de la tabla en MySQL que almacena el inventario sea de tipo `LONGTEXT` o `LONGBLOB`, ya que los NBT de los mods pueden exceder fácilmente los límites estándar de un VARCHAR.